

Interview Simulator — Work Split v3 (Steps 1-10, Full MVP Scope)

What changed from v2:

1. **Step 1 is rewritten** — official Reddit API registration is slow and increasingly gated, so instead of building custom scrapers against authenticated APIs, both of you evaluate and reuse *existing* scraper tools/libraries for Reddit and Medium. Less time spent on ingestion plumbing, more on the AI-heavy steps.
2. **Steps 7-10 are new** — they turn the RAG backend from v2 into the actual Version 1 MVP described in the product doc: auth, resume parsing, interview modes, voice, the adaptive engine, live coding intelligence, dynamic feedback, and the dashboard.

The same philosophy holds throughout: on every AI/ML-heavy step, **both people write a prompt, call a model, and validate results independently** before merging. No one gets to sit out the AI parts.

STEP 1 — Data Ingestion Setup (Revised: existing scrapers, no Reddit API wait)

Shared Goal

Two working data feeds (Reddit + Medium/secondary) using existing scraper tools, not custom-built from scratch — plus an agreed data-quality standard.

Person A — Reddit, no-auth route

1. Skip formal Reddit API app registration/approval. Use Reddit's public read-only JSON endpoints (e.g. `https://www.reddit.com/r/<subreddit>/search.json?q=...`) via `requests`/`httpx`, or an existing open-source wrapper that already handles this (check 1-2 GitHub repos before writing anything custom — e.g. lightweight PRAW read-only mode, or existing "reddit scraper" packages on PyPI/GitHub).
2. Target subreddits: `csMajors`, `cscareerquestions`.
3. Pull posts matching "interview experience", "onsite", "OA" — same fields as before: `title`, `selftext`, `subreddit`, `created_utc`, `permalink`, `id`.
4. Save 15+ posts to CSV/JSON.
5. Note rate limits / polite delay between requests — public endpoints throttle harder than authenticated ones.

Person B — Medium (and similar), existing scraper

1. Before writing custom scraping code, evaluate 2-3 existing options: open-source Medium scraper repos on GitHub, or scraper-as-a-service tools (e.g. Apify's Medium actor, ScraperAPI, similar). Pick whichever gets you working data fastest.
2. Target interview-experience tags/publications on Medium.
3. Same field extraction pattern as Person A.

4. Save 15+ articles to CSV/JSON.

🧡 Together

Merge both batches. Manually review 10 combined items as a pair — Company? Role? Year? OA? Rounds? Topics? Difficulty? Outcome? Agree on the final raw field list: `(source, source_url, raw_text, company_guess, role_guess, submitted_at)`. Document in your report which existing tool you used for each source and why — this replaces "we built a scraper" with "we evaluated and integrated an existing tool," which is a legitimate engineering decision worth writing up.

✅ **Done when:** both feeds return real data without either of you waiting on API approval, you have 30+ combined items, and a written field list.

STEP 2 — Central Database + Data Quality

(unchanged from v2 — still applies regardless of which scraping tools you used)

🎯 Shared Goal

All raw data lands in one Postgres table; you understand your data sources before trusting them.

👤 Person A — Database + Reddit pipeline

1. Set up Supabase/Neon Postgres, enable `pgvector` now.
2. Create `raw_reports` table.
3. Push Step 1's combined Reddit output into the table.

👤 Person B — Form pipeline + Medium validation

1. Pull Google Form responses (via linked Sheet / Sheets API) into the same table.
2. Manually read 15–20 Medium articles under interview-experience tags — log whether each actually has company/role/rounds/topics or is generic advice. Decide: worth keeping in the pipeline, or filter out?

```
sql

CREATE TABLE raw_reports (
  id SERIAL PRIMARY KEY,
  source TEXT NOT NULL,
  source_url TEXT,
  raw_text TEXT NOT NULL,
  company_guess TEXT,
  role_guess TEXT,
  submitted_at TIMESTAMP,
  processed BOOLEAN DEFAULT FALSE
);
```

🧡 Together

Query the table, eyeball 10 rows from each source side by side. Agree on the Medium go/no-go filter together.

✅ **Done when:** `raw_reports` has real rows from Reddit, Form, and Medium (filtered), and you've both looked at real rows from every source.

STEP 3 — LLM Structured Extraction

(unchanged from v2)

🎯 Shared Goal

Turn messy raw text into structured JSON.

👤 Person A — Identity fields

1. Schema for `company`, `role`, `year`, `outcome`.
2. Extraction prompt on Step 1's Reddit batch (15+ posts), validated with `pydantic`.
3. Check accuracy by reading the posts yourself.

👤 Person B — Interview-structure fields

1. Schema for `rounds` (array of `{type, topics, difficulty}`) and `tone`.
2. Extraction prompt on the Google Form batch (10 responses) — cleaner data, good for testing nested extraction first.
3. Check accuracy the same way.

```
sql

CREATE TABLE structured_reports (
  id SERIAL PRIMARY KEY,
  raw_report_id INT REFERENCES raw_reports(id),
  company TEXT, role TEXT, year INT,
  rounds JSONB, tone TEXT, outcome TEXT
);
```

🧡 Together

Merge both prompts into a single combined extraction call. Test on 10 combined examples, fix whichever fields perform worst. Then split remaining work:

- **Person A:** fallback hierarchy (round-specific → role-specific → company-generic → global default) for sparse data.
- **Person B:** `structured_reports` table + insert logic linking back to `raw_reports`.

✅ **Done when:** one merged, tested extraction prompt both of you validated, structured data flowing in with fallback logic.

STEP 4 — Search & Analytics Layer

(unchanged from v2)

Person A — Topic search + indexing

1. GIN index on `rounds` JSONB.
2. Inverted-index-style query: topic → matching report IDs.
3. "Most common topics per company" query.

Person B — Aggregation + dashboard-ready stats

1. Company → topic-frequency lookup.
2. Round-count distribution + difficulty-trend-over-time queries.
3. (Optional) Trie for company/role autocomplete.

Together

Run real queries together, sanity-check against Step 2's eyeballed rows.

✅ **Done when:** topic search returns correct IDs and dashboard stat functions return real numbers.

STEP 5 — Embeddings + Vector Search

(unchanged from v2)

Person A — Embed and store

1. Summary string per report: `f"{company} {role} {round_type} covering {topics}"`.
2. Call an embedding model on each summary.
3. Store in `embedding vector(768)` via pgvector, as a batch job.

Person B — Embed queries + search

1. Query-side embedding function.
2. Similarity search: `SELECT * FROM structured_reports ORDER BY embedding <-> $1 LIMIT 5;`
3. Test with real queries, log 2-3 good/bad matches.

Together

Tune together if results are weak — A adjusts summary text, B re-tests.

✅ **Done when:** every report has an embedding, test queries return relevant results.

STEP 6 — RAG-Powered Q&A + Mock Interview Engine (baseline)

(unchanged from v2 — this becomes the retrieval core that Step 8 plugs into)

Person A — Mock interview engine (baseline)

1. Retrieval call: embed company/role/round selection, pull top N reports.

2. Interview-conducting prompt grounded in retrieved tone/topics/frequency.
3. Multi-turn conversation history passed each turn.
4. Test end-to-end in a script/notebook.

Person B — Prep Q&A + feedback engine (baseline)

1. Open Q&A generation prompt, cite patterns from retrieved reports.
2. Test with real prep questions, log a grounding example and a drift example.
3. Feedback-scoring prompt (STAR/depth/clarity, 1-5 each) + one tip per category.
4. Test on 2-3 transcripts from Person A's engine.

Together

Run the full loop: company → A's engine interviews → B's function scores it.

✅ **Done when:** the baseline mock interview + feedback cycle works end to end.

STEP 7 — Auth, Profiles & Resume Intelligence (NEW — Mandatory ★)

Shared Goal

Users can sign up, log in, and upload a resume that gets parsed into structured data every interview will draw from.

Person A — Auth & Profile

1. Set up auth (Supabase Auth / Clerk / NextAuth) — sign up, login, sessions.
2. Build `users` / `profiles` table and an `interview_history` table (references sessions from Step 10).
3. Basic profile UI: view past interviews, edit profile.

Person B — Resume Upload & Parsing

1. PDF upload endpoint.
2. Extract text (e.g. `pdfplumber`), then an LLM extraction prompt (same pattern as Step 3) pulling: Education, Experience, Projects, Skills, Leadership, Achievements.
3. Validate with a `pydantic` schema; store as structured JSON linked to the user.
4. Test on 10+ real resumes, log where extraction breaks (multi-column layouts, tables, etc.).

Together

Wire parsed resume data into the profile so every interview session can reference it.

Confirm the flow end to end: sign up → upload resume → see correctly structured data.

✅ **Done when:** a user can sign up, upload a resume, and see accurate structured resume data tied to their account.

STEP 8 — Interview Configuration, Modes & Voice Conversation (*NEW — Voice is Mandatory* ★)

Shared Goal

User configures an interview (Company → Role → Level → Round) and has a natural **voice** conversation, across all four modes.

Person A — Config + RAG hookup + Technical/HR modes

1. Config flow UI: Company → Role → Experience Level → Round.
2. Wire selections into Step 6's retrieval call (this is your "company-specific interview generation").
3. Technical Discussion mode: resume / DBMS / OS / CN / OOP / ML questions, no editor.
4. HR Round mode: resume-driven behavioural questions (tell me about yourself, why us, leadership, internship).

Person B — Voice pipeline + Coding/ML modes

1. Voice pipeline: STT for the candidate, TTS for the interviewer, real-time turn-taking (no typing).
2. Coding Round mode shell: Monaco editor + problem statement + voice interviewer (live reasoning comes in Step 9).
3. ML Round mode: resume → ML theory → project discussion → case studies → model selection/trade-offs.

Together

Run one full voice-driven mock interview per mode (4 total). Confirm questions actually reflect the company/role selected, not generic filler.

✅ **Done when:** all four modes run as real voice conversations grounded in company-specific RAG data.

STEP 9 — Adaptive Interview Engine & Live Coding Intelligence (*NEW — Core Differentiator* ★ ★ ★)

Shared Goal

The interviewer listens, decides when to interrupt or follow up, and — in coding rounds — reasons about code as it's written rather than only at the end.

Person A — Adaptive Interview Engine

1. Design the decision loop: Observe → Think → Should I interrupt? → Ask/Wait → Listen → Follow-up?
2. Implement as a controller wrapping Step 6's conversation prompt — track what's been covered, answer depth, confidence signals.
3. Test: produce interviews where the engine sometimes stays silent, sometimes asks 2–3 follow-ups. Log a few transcripts showing this variation.

Person B — Live Coding Intelligence

1. Code-diff watcher: debounced snapshots of editor state over time.
2. Feed diffs to a prompt reasoning about algorithm choice, optimisation, bugs — trigger a "why?" on meaningful changes, not just at submission.
3. Test on 3-5 real coding sessions; log false positives (interrupting too often) and tune sensitivity.

Together

Integrate live coding intelligence as an input into the adaptive engine's decision loop specifically for coding rounds, so the two systems act as one there. Run a full coding interview together and refine thresholds.

✅ **Done when:** the interviewer visibly adapts (varies follow-ups, sometimes says nothing) and, in coding rounds, asks "why" mid-solution based on real code changes.

STEP 10 — Dynamic Feedback & Dashboard (*NEW*)

Shared Goal

Every interview produces multi-dimensional feedback, and users can track progress over time.

Person A — Dynamic Feedback engine

1. Extend Step 6's feedback prompt to score: Technical ability, Communication, Problem solving, Optimisation, Debugging, Confidence, Resume discussion, Behavioural, Overall recommendation.
2. Test on transcripts from Step 8/9 sessions across all 4 modes — adjust which dimensions apply per mode (e.g. Optimisation/Debugging mainly matter for Coding).
3. Store feedback per session, linked to the user.

Person B — Dashboard

1. Queries: interviews completed, strengths/weak areas (aggregated from feedback scores), company readiness, improvement over time.
2. Dashboard UI (trend charts) consuming those queries.
3. Test with a user who has 5+ completed interviews across different companies.

Together

Run the full product loop: complete an interview → see dynamic feedback → see it reflected on the dashboard. **This is your core end-to-end demo.**

✅ **Done when:** the dashboard accurately reflects real interview history and feedback trends.

Version 2 (Deferred — not needed to launch)

- Full hiring pipeline simulation (OA → Technical → Hiring Manager → HR with memory across rounds).
- Multi-language coding support beyond the initial language(s).
- Pair programming mode.
- Multiple interviewer personalities (strict, supportive, fast-paced, etc.).
- Team/panel interviews.
- Whiteboard & system design canvas.
- Recruiter dashboard for companies.
- AI-generated personalized study plans based on interview history.

Summary: Who Owns What

Step	Person A	Person B
1	Reddit (existing scraper, no-auth)	Medium (existing scraper)
2	DB setup + Reddit pipeline	Form pipeline + Medium validation
3	Identity-field extraction + fallback logic	Round-structure extraction + storage
4	Topic search + indexing	Dashboard aggregation
5	Embed & store report data	Embed queries + search
6	Mock interview engine (baseline)	Prep Q&A + feedback engine (baseline)
7	Auth & profile	Resume upload & parsing
8	Config + RAG hookup + Technical/HR modes	Voice pipeline + Coding/ML modes
9	Adaptive interview engine	Live coding intelligence
10	Dynamic feedback engine	Dashboard

The balance holds end to end: in every AI/ML-heavy step (3, 5, 6, 7's resume parsing, 9), both people write a prompt, call a model, and validate results independently before merging.